

ParcoursSup

Un Apprentissage Par Problème (APP) destiné aux étudiants
du module Algorithmique avancée 1

Henri HAGBE





Situation-problème

Demain matin, une commission d'enquête rend public un rapport intitulé : « Parcoursup, manque de transparence et reproduction des inégalités à grande échelle ». Les premières lignes de ce rapport font l'effet d'une bombe.

La pression médiatique monte d'un cran quand un député demande une commission d'enquête parlementaire. En urgence, le ministère de l'enseignement supérieur mandate une équipe "Algo + Audit".

C'est là que vous intervenez.

Vous êtes une équipe de développeurs qui travaille pour le service public. Votre mission : produire un prototype crédible capable de recalculer, à grande échelle, les ordres d'appel des formations (c'est-à-dire l'ordre dans lequel les propositions d'admission seraient envoyées), puis démontrer par des tests que votre version est plus robuste et plus auditable que ce qui existe actuellement.

Attention : Vous l'avez-vous-même vécu, il n'existe pas de "classement national unique" des élèves. Chaque formation possède sa propre liste de candidats. En revanche, le calcul des ordres d'appel est un traitement national massif : des milliers de listes à produire sur des volumes énormes, dans un temps limité.

Travail demandé

Votre mission consiste donc à concevoir un outil qui permet de :

1. Charger des données :

- Lire les données à partir d'un fichier csv contenant toutes les informations possibles sur les vœux de plusieurs centaines de milliers d'étudiants
- Introduire les données dans une structure adaptée pour faciliter la manipulation et la recherche

2. Mettre les données au bon format :

- Implémenter des fonctions intermédiaires permettant de :
 - Organiser les données dans une structure claire et compréhensible.
 - Afficher les données complètes concernant une candidature.
 - Récupérer les informations d'un étudiant donné grâce à son identifiant.

3. Trier et afficher les données :

- Implémenter et adapter les algorithmes de tri du cours pour classer les vœux des étudiants selon la filière choisie.

4. Comprendre et répliquer le mécanisme de ParcoursSup (simplifié !)

- Regrouper les candidatures par formation (program_id), calculer un ordre d'appel par formation, puis extraire admis (capacité) + liste d'attente.

5. Proposer 2 POLICIES

- Une base de tri simple et auditable sur les données fournies.
- Une version corrigée visant à réduire une faille mesurable (ex æquo arbitraires, micro-écarts mal calculés, performance non optimale).

6. Mettre en œuvre une stratégie de tri adaptée aux données massives (tri fusion / tri rapide / tri casier si pertinent) et mesurer les performances sur des volumes importants.

7. Réaliser des tests d'audit sur au moins 2 failles du système (avant/après correction) et produire des preuves chiffrées :

- Que se passe-t-il en cas de ...
 - Ex æquo arbitraires (tie-break implicite)
 - Micro-écarts trop fins (au centième de points)
 - Performance irréaliste (un tri en n^2 pour des millions de données !)

8. Préparer un court rapport (2–4 pages) et une démonstration orale (format “audition devant une commission”).

9. [Bonus] Comparer les algorithmes de tri entre eux :

- Permettre aux testeurs de :



- Mener une étude comparative des algorithmes de tris (fusion, rapide, casier et insertion) avec affichage graphique à la clé grâce à la librairie Python **matplotlib**. Les élèves traceront une courbe de la durée des différents tris en fonction de la taille des données à trier.

Données fournies : structure et sens des champs

Chaque ligne représente une candidature (un vœu) : un candidat postule à une formation. L'objectif est de produire un ordre d'appel par formation.

candidate_id : Identifiant du candidat (entier). Sert aussi de tie-break final (répérable, non ambigu).

program_id : Identifiant de la formation (entier). Permet de regrouper les candidatures par formation.

score : Score de candidature (entier borné, ex. 0..10000). Sert à classer (baseline) ; borné ce qui est utile pour le tri casier.

timestamp : Ordre/heure de dépôt du vœu (entier). Tie-break naturel pour l'audit : à score égal, qui était "devant" ?

is_scholarship : 0/1 : candidat boursier (bourse sur critères sociaux). Sert à tester des règles d'équité (quota, priorité à égalité) et à mesurer l'impact (% boursiers admis vs % boursiers candidats).

hs_id : Identifiant du lycée d'origine (code). Sert à l'audit statistique : diversité des lycées représentés parmi les admis, concentration sur quelques lycées, etc. (outil d'analyse, pas un jugement).

Les ressources pour traiter la situation-problème

- Le cours 3 et le TD 3 « Algorithmes de tri : Tri par insertion, Tri par sélection, Tri par bulles ».
- Le cours 4 et le TD 4 « Algorithmes de tri 2 : Tri par fusion, tri rapide »
- Le cours des « fichiers » (cours du 1^{er} semestre : module « Python »)
- Optionnel : Le cours des « interfaces graphiques Tkinter » (cours du 1^{er} semestre : module « Python »)
- Le jeu de données disponible en lien et sur Moodle

Objectifs d'apprentissage de l'APP (AAV) :

- Comprendre et maîtriser les algorithmes de tri.
- Tester et comparer les algorithmes de tri et pouvoir identifier l'algorithme le plus rapide.
- Se répartir les rôles et les tâches au sein de l'APP.
- Commenter de manière pertinente les programmes.
- Synthétiser les travaux du groupe.
- Se servir du langage Python pour traiter un projet d'envergure sur des données gouvernementales.
- Importer, utiliser et tester un jeu de données à partir d'un fichier CSV.
- Comprendre le lien entre la complexité théorique calculée en classe et le temps effectif mis par les différents tris étudiés cette année